Competitive Programming

Section 42: Advanced Data Structures Mixed Contest

Competitive Programming Course

2026-10-05

Table of contents

1 A mixed contest, using established tools 3

2 Prerequisites and contest contract 4

3 Model the operations before choosing a structure 5

4 Worked diagnosis: a tree-shaped input 6

5 Worked diagnosis: order matters in lazy updates 7

6 Worked diagnosis: static questions may be reordered 8

7 Proof and cost accounting 9

8 C++ and Python implementation habits 10

9 Exercises 11

10 Testing and submission checklist 12

1 A mixed contest, using established tools

This section is a contest checkpoint. It combines techniques already taught through section 41; it does not introduce a new data structure. Difficulty comes from choosing a representation, composing known operations, handling boundaries, and proving that the combination answers the original question.

Heavy-light decomposition belongs to section 68, and persistent structures belong to section 67. Neither is a prerequisite here. In particular, a static binary-lifting aggregate does not support arbitrary value changes efficiently. A segment tree handles intervals in its base array; it needs a valid mapping before it can answer a question about a tree.

2 Prerequisites and contest contract

Established material	Earlier section
Prefix sums, difference arrays	4
Sorting, interval sweeps, coordinate compression, offline ordering	5, 11, 13
DFS and rooted trees	16, 26
DSU and Kruskal minimum spanning trees	18
Sliding windows	6
Fenwick prefix sums and point updates	36
Segment-tree aggregates and lazy propagation	37
Static aggregates and binary lifting	38
Count-guided segment-tree descent	40
Euler entry intervals and LCA	41

Use these operations as building blocks. You may derive a new reduction from them, but no exercise requires HLD, persistence, rollback DSU, merge-sort trees, or multidimensional Fenwick trees. All offline exercises expose the complete input before processing.

Attempt A–F first, then G–L. The second half asks for more combinations of familiar tools rather than new structures. Do not open the editorial until you have written an operation contract and tried a slow correct solution.

3 Model the operations before choosing a structure

Write down what changes and what stays fixed. An assignment is different from an addition. A static set of weighted points permits a sweep; interleaved point insertions and rectangle queries would require a different argument. An edge-insertion graph fits ordinary DSU; deletion does not.

Separate the object being queried from the stored summary:

Question	Possible stored information
sum over an interval	segment sum or two prefix sums
maximum over an interval	segment maximum
number of values up to a threshold	frequency prefix
k-th present value	counts in left and right segments
whether two vertices are connected	component representatives

A summary is sufficient only if it can answer every operation. In particular, subtracting two prefix maxima does not recover an interval maximum. Addition has an inverse; maximum does not.

4 Worked diagnosis: a tree-shaped input

Root a small tree with edges `1-2`, `1-3`, `3-4`, and `3-5`. A preorder can be:

```
vertex order:    1  2  3  4  5
entry position:  0  1  2  3  4
subtree(3):           [2, 5)
```

As established in section 41, DFS enters all descendants of a vertex before leaving its subtree. Record `tin[u]` on entry and `tout[u]` after the last subtree vertex. Its vertices occupy exactly `[tin[u], tout[u])`.

The important distinction is the query's set of vertices. The path `2-1-3-4` has positions `1,0,2,3` in traversal order. General paths do not have a guaranteed single contiguous preorder interval. Never replace a path by the interval between its endpoints.

For a subtree operation, the established mapping suffices. A point assignment changes one Euler position; a subtree addition changes one interval. Decide which summary to store only after establishing the mapping. A one-vertex tree has interval `[0,1)`, and the root's subtree is the whole array.

5 Worked diagnosis: order matters in lazy updates

Recall the range-update invariant from section 37: a node's summary already includes every update covering the entire node, while a lazy tag records work still owed to its children. Push before inspecting those children.

For a length-three segment initially containing `2,4,6`, compare:

Operation order	Result	Sum
assign 7, then add -2	`5,5,5`	15
add -2, then assign 7	`7,7,7`	21

A pending assignment needs a separate presence flag, because assigning zero is a valid update. A later assignment replaces earlier pending work. A later addition changes an existing pending assignment, or accumulates into an addition tag when there is no assignment. These are consequences of the operation contract, rather than a new structure.

For maximum with uniform addition, every value shifts by the same amount: `max(a_i + d) = max(a_i) + d`. Thus the familiar lazy range-add mechanism can maintain a maximum summary instead of a sum. For sum, add `d * length`; for maximum, add `d`. State which one your node stores.

6 Worked diagnosis: static questions may be reordered

Suppose fixed objects have values `2,5,5` and queries ask how many are at most `4` and at most `5`. Sort objects by value and queries by threshold. Before answering threshold `T`, insert precisely the objects with value at most `T`. The counts are `1` and `3`; both equal-valued objects must be inserted before the second answer.

The active objects can carry positions, coordinates, or weights. A one- dimensional Fenwick tree summarizes the relevant coordinate of the active set. Recover original query order with query identifiers. An inclusive interval `[lo,hi]` is the difference between thresholds `hi` and `lo-1` for integers.

This is the sorting-and-sweep pattern from sections 13 and 36. Static input allows the reordering. If updates alter the objects between queries, sorting queries alone would discard their time-dependent meaning.

7 Proof and cost accounting

For every proposed combination, prove these claims separately:

1. The representation describes exactly the requested objects.
2. Each stored number has a precise meaning.
3. Each update preserves that meaning.
4. Combining summaries counts each requested object exactly once.
5. Preprocessing, all operations, and memory fit the constraints.

For a sweep, state which objects have been inserted before each query. For a rooted-tree mapping, prove subtree contiguity. For a bottleneck reduction, prove why an optimal forest path answers the original graph query. For lazy updates, distinguish what the parent summary includes from what its children still need.

Sorting can dominate an otherwise logarithmic query structure. Coordinate compression preserves order and equality, not distances. Use lower and upper bounds correctly even when a query boundary is absent from the coordinates.

8 C++ and Python implementation habits

Use signed 64-bit sums in C++. At most `200000` values of magnitude `10^9` already require more than 32 bits; repeated additions can increase the total. Python integers handle these sums, but parsing and repeated function calls still affect running time.

Traverse large input trees iteratively in both languages: a chain may have `200000` vertices. Entry/exit stack events reproduce recursive DFS without a call stack proportional to the tree height. Segment-tree recursion is only logarithmic in the array length. Keep one interval convention throughout; convert explicitly if a template uses inclusive 1-based intervals.

For maximum, use a neutral value below every possible actual value, not zero. For counts, prevent removal of absent elements so counts remain nonnegative. For DSU, use union by size and path compression as in section 18.

9 Exercises

Problem	Title
A	Dynamic Order Statistics
B	Range Assign and Add
C	Static Rectangle Count
D	Nested Ranges Count
E	Subtree Add, Point Query
F	Dynamic Subtree Maximum
G	Subtree Value Count
H	Incremental Connectivity
I	Rerooted Subtree Sums
J	Minimum Bottleneck Routes
K	Sliding Window Inversions
L	Subtree Add, Subtree Maximum

For each problem, write a slow oracle and name the invariant before coding. The editorial gives problem-specific reductions after your attempt.

10 Testing and submission checklist

Feature	Targeted checks
frequency selection	duplicates, empty multiset, invalid rank
lazy composition	assignment to zero, add then assign, nested intervals
offline sweep	equal coordinates, boundaries absent from compression
interval containment	identical intervals and equal endpoints
tree intervals	single vertex, root, leaf, chain, star
changing roots	current-root query, ancestor complement, unrelated branch
bottleneck routes	disconnected graph, cycles, zero weights, same endpoint
window inversions	equal values, k=1, decreasing window, 64-bit counts
connectivity	repeated edges, self-loops, separate components

Compare small random cases with direct array scans or graph searches. Large chain tests expose accidental recursive DFS and subtree scans; large sweeps expose per-query scans. Avoid relying on samples to establish complexity.

Run your private submissions with the section checker. To verify the public reference solutions instead:

```
CP_TARGET=solution python3 sections/42_advanced_data_structures_mixed/check.py --keep-going
```

Before submitting, verify the operation contract, inclusive boundaries, neutral element, update order, total complexity, and maximum numeric size.